

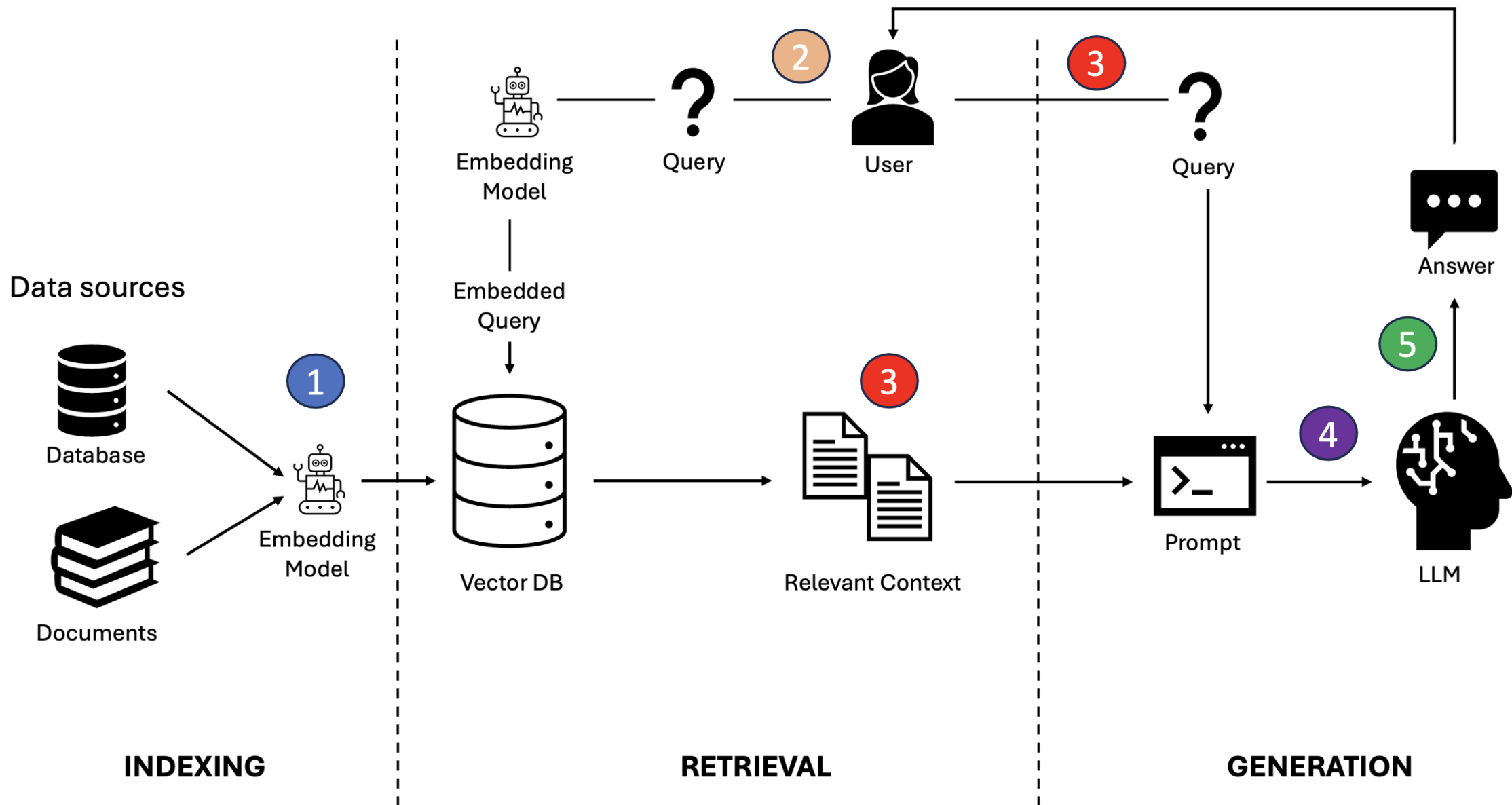


RAG - Retrieval Augmented Generation

- Lezione 19 - Prof. Giuseppe Fenza

- Problema iniziale:
 - Gli LLMs hanno conoscenza e capacità limitate ai loro dati di addestramento.
 - Ciò può causare risposte imprecise e/o allucinazioni gravi.
- Soluzione al problema:
 - La Retrieval-Augmented Generation (RAG) è un paradigma che permette di risolvere (in parte) il problema descritto fornendo conoscenza aggiuntiva (contesto) al modello.
 - Il LLM avrà a disposizione conoscenza aggiornata e mirata per rispondere ad una specifica domanda.
- Come funziona?:
 - Solitamente, la RAG, nella sua versione più semplice, si compone di 3 fasi:
 1. *Indexing* (Indicizzazione);
 2. *Retrieval* (Recupero);
 3. *Generation* (Generazione).

RAG



- INDEXING (Indicizzazione):
 - Il processo inizia con la creazione di un **indice** da fonti esterne (ad esempio, file PDF).
 - L'**indice** è una struttura dati ottimizzata che permette al modulo di Retrieval di cercare e trovare rapidamente i documenti più rilevanti.
 - L'indicizzazione implica la preparazione dei dati normalizzando il testo attraverso i processi come *stemming* e *tokenizzazione*.
 - Il testo pre-elaborato viene suddiviso in *chunks* (pezzi di testo composti da un numero determinato di parole).
 - I chunks vengono trasformati in *rappresentazioni vettoriali semantiche* mediante modelli linguistici pre-addestrati (*modelli di embedding*).
 - Queste rappresentazioni vettoriali sono archiviate in un *Vector Database* per consentire un recupero più efficace e rapido.

- RETRIEVAL (Recupero):
 - In questa fase viene utilizzata la **query** fornita dall'utente per recuperare il contesto rilevante (utile per rispondere ad essa) dal Vector Database.
 - La query viene trasformata in una rappresentazione vettoriale semantica usando lo stesso modello di embedding impiegato durante la fase di indicizzazione.
 - Vengono poi calcolate le **similarità semantiche** (attraverso **distanze vettoriali**, come: distanza euclidea, distanza coseno, ecc.) tra le rappresentazioni vettoriali dei chunks (calcolate nella fase precedente) e quella della query.
 - In questo modo sono recuperati i **K** chunks più simili alla query (e quindi più rilevanti per la successiva risposta).

- GENERATION (Generazione):
 - Una volta recuperato il contesto rilevante, la query e queste informazioni recuperate vengono integrate all'interno di un prompt da fornire all'LLM.
 - L'LLM sarà in grado di fornire una risposta aggiornata per il task desiderato.

- I problemi per sistemi di RAG semplici possono essere riassunti come segue:
 - **Problemi di Retrieval:** Selezione di chunks non pertinenti (irrelevant chunks) e mancanza di informazioni importanti a causa della mancanza di precisione.
 - **Problemi di Generation:** Il modello può produrre risposte il cui contenuto non è supportato dal contesto recuperato (allucinazioni).
 - **Problemi di Augmentation:** Se vengono recuperate informazioni simili da fonti diverse, la risposta può risultare ripetitiva a causa di informazioni ridondanti. Inoltre, i modelli potrebbero fare affidamento esclusivamente sulle informazioni recuperate, portando a ripetere quel contenuto recuperato senza aggiungere nulla di nuovo o interessante.

- Per risolvere i problemi visti in precedenza si possono adottare diverse strategie. Ad esempio, si ci può concentrare sul miglioramento della qualità del retrieval (recupero), impiegando strategie di **pre-retrieval** e **post-retrieval**.
- **Pre-Retrieval**, in questa fase, l'obiettivo principale è ottimizzare la struttura di indicizzazione e la query originale:
 - **Ottimizzazione Indicizzazione**: Ha lo scopo di migliorare la qualità del contenuto indicizzato, attraverso, per esempio, l'aggiunta di metadati e migliore granularità dei dati (livello di dettaglio del dato).
 - **Ottimizzazione Query**: Mira a rendere la domanda originale dell'utente più chiara e adatta al compito di retrieval. I metodi comuni includono la *riscrizione della query* (query rewriting), la *trasformazione della query* e l'*espansione della query* (sostituire o aggiungere alla query sinonimi del termine originale, e/o usare più versioni diverse della stessa query, ma pertinenti) tutto ciò per migliorare i risultati di ricerca, ottenendo una maggiore precisione e completezza delle informazioni.

- **Post-Retrieval**, una volta recuperato il contesto pertinente, è fondamentale integrarlo efficacemente con la query. I metodi principali nel processo di post-retrieval includono:
 - **Riordinamento dei Chunks:** Riordino delle informazioni recuperate per ottimizzare la posizione del contenuto più rilevante.
 - **Compressione del Contesto:** L'inserimento di tutti i documenti recuperati direttamente nell' LLM può portare a un sovraccarico di informazioni. Per mitigare ciò, gli sforzi di post-retrieval si concentrano sulla selezione delle informazioni essenziali, sottolineando le sezioni critiche e accorciando il contesto da elaborare.

RAG – Tecniche avanzate



- La RAG può diventare anche più complessa, configurandosi in un sistema composto da diversi moduli per offrire una maggiore adattabilità e scalabilità, nonché un miglioramento delle risposte.
- Vediamo alcuni dei moduli più diffusi per i sistemi RAG:
 - **Modulo di Ricerca (*Search module*)**: Si adatta a scenari specifici, consentendo **ricerche dirette** in varie fonti di dati (motori di ricerca, database, *knowledge graph*).
 - **RAG-Fusion**: Affronta le limitazioni di ricerca tradizionali impiegando una **strategia multi-query** che espande le *query* utente in diverse prospettive, utilizzando ricerche vettoriali parallele e un riordinamento intelligente.
 - **Modulo di Memoria (*Memory module*)**: Sfrutta la memoria dell'LLM per **guidare il retrieval**.
 - **Instradamento (*Routing*)**: Nel sistema RAG, naviga attraverso diverse fonti di dati, selezionando il **percorso ottimale** per una *query* (riepilogo, ricerca specifica in database, fusione di flussi di informazioni).
 - **Modulo Adattatore di Compito (*Task Adapter module*)**: Adatta il RAG a **vari compiti a valle**, automatizzando il *retrieval* del *prompt* e creando *retriever* specifici per il compito.

RAG – Tecniche avanzate – Search Module



Modulo di Ricerca (*Search module*): Si adatta a scenari specifici, consentendo **ricerche dirette** in varie fonti di dati (motori di ricerca, database, *knowledge graph*).

Un esempio è KnowledgeGPT¹, un framework progettato per integrare gli LLM con le **Knowledge Bases (KB)**, gestendo sia il recupero che l'archiviazione della conoscenza.

Il cuore del sistema è un processo di recupero a tre fasi che utilizza il **Program of Thought (PoT) prompting**: per una data *query*, l'LLM genera codice Python (il Program of Thought) che contiene chiamate a funzioni predefinite per interagire con le KB.

Questo codice viene quindi eseguito per facilitare la **ricerca** e risolvere le ambiguità delle entità tramite *entity linking*, permettendo infine all'LLM di sintetizzare la conoscenza recuperata per generare una risposta accurata.

¹ Wang, X., Yang, Q., Qiu, Y., Liang, J., He, Q., Gu, Z., ... & Wang, W. (2023). Knowledgpt: Enhancing large language models with retrieval and storage access on knowledge bases. *arXiv preprint arXiv:2308.11761*.

RAG – Tecniche avanzate – RAG-Fusion



RAG-Fusion¹ : Affronta le limitazioni di ricerca tradizionali impiegando una **strategia multi-query** che espande le *query* utente in diverse prospettive, utilizzando ricerche vettoriali parallele e un riordinamento intelligente.

- Il processo inizia con un LLM che, partendo dalla query originale dell'utente, genera più query di ricerca correlate.
- Per tutte queste query (originale + generate) si esegue il recupero di documenti tramite ricerca vettoriale nel database.
- Infine, i diversi set di risultati vengono combinati e riordinati in un'unica lista utilizzando un specifico che assegna un punteggio basato sulla posizione del documento in ciascuna lista.
- I documenti così fusi e riordinati vengono inviati all'LLM per generare una risposta finale più completa e contestualizzata.

¹ Rackauckas, Z. (2024). Rag-fusion: a new take on retrieval-augmented generation. *arXiv preprint arXiv:2402.03367*.

RAG – Tecniche avanzate – Memory Module



Modulo di Memoria (*Memory module*): Sfrutta la memoria del LLM per **guidare il *retrieval***.

Un metodo correlato¹ si basa sull'integrazione di un meccanismo di autocorrezione che precede la fase di recupero standard.

Il cuore del sistema è il **Memory module** (o Self-Memory), che sfrutta le conoscenze parametriche intrinseche dell'LLM per guidare e ottimizzare la ricerca esterna.

Prima di interagire con il database, l'LLM, attraverso questo modulo, genera internamente query di ricerca alternative, riformulazioni della query, o identifica di punti ciechi (**Self-Correction/Self-Refinement**).

Queste query ottimizzate e diversificate vengono poi **inviate al retriever** per recuperare un contesto più ricco e pertinente dall'archivio esterno (ad esempio, un vector store).

Infine, il testo generato viene migliorato usando i risultati recuperati più precisi, garantendo che la risposta finale sia non solo ancorata a prove esterne, ma anche guidata da una comprensione più sfumata ottenuta tramite il modulo di memoria.

¹ Cheng, X., Luo, D., Chen, X., Liu, L., Zhao, D., & Yan, R. (2023). Lift yourself up: Retrieval-augmented text generation with self-memory. *Advances in Neural Information Processing Systems*, 36, 43780-43799.

RAG – Tecniche avanzate – Routing



Routing: è una tecnica avanzata utilizzata per instradare in modo dinamico una query in input a diversi indici o pipeline RAG specializzate, al fine di migliorare l'efficienza e la pertinenza nel recupero di informazioni in sistemi con vaste knowledge base o molte fonti di dati.

Routing Logico (Basato su LLM)

- Affida la scelta della risorsa, da cui recuperare informazioni, ad un **LLM** che agisce da router. L'LLM analizza la **query ambigua** dell'utente e decide a quale sorgente (es. database HR o Finanziario) indirizzarla. È eccellente nel gestire **linguaggio ambiguo** o errori di battitura. Lo svantaggio è la potenziale ignoranza di **termini aziendali proprietari** o la sua inaffidabilità su conoscenze di nicchia.

Routing Semantico (Basato su Similarità)

- Utilizza la **similarità vettoriale (embedding)** tra la query dell'utente e le descrizioni (prompt) di ciascuna sorgente per effettuare l'instradamento. La query viene inviata alla sorgente il cui embedding è **semanticamente più vicino**. È ideale per gestire **termini altamente specifici** dell'organizzazione, poiché la similarità è misurata sugli embedding. La limitazione principale è che l'accuratezza può peggiorare se i prompt descrittivi sono troppo lunghi.

RAG – Tecniche avanzate – Task Adapter



Task Adapter module: Adatta il RAG a **vari compiti a valle**, automatizzando il *retrieval* del *prompt* e creando *retriever* specifici per il compito.

È un modulo «neurone leggero» aggiunto ad un LLM per adattare l'LLM stesso ad un nuovo compito specifico, addestrando **solo i suoi pesi** e mantenendo congelato il modello base.

Il suo scopo principale è permettere di adattare un modello pre-addestrato (che ha appreso conoscenze generali su enormi quantità di dati) ad un compito specifico (come la classificazione di sentiment, la traduzione, o la risposta a domande) senza dover effettuare un costoso e dispendioso fine-tuning dell'intero modello.

¹ Cheng, D., Huang, S., Bi, J., Zhan, Y., Liu, J., Wang, Y., ... & Zhang, Q. (2023, December). Uprise: Universal prompt retrieval for improving zero-shot evaluation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing* (pp. 12318-12337).

RAG – Tecniche avanzate – Task Adapter



UPRISE¹ (Universal Prompt Retrieval for Improving Zero-Shot Evaluation) è una metodologia che utilizza un **retriever «leggero»** per **selezionare automaticamente e dinamicamente i prompt** più efficaci da un database, in base all'input di un compito.

Questo approccio è progettato per **migliorare le prestazioni zero-shot** degli LLMs senza la necessità di un fine-tuning completo del modello base.

In questo contesto (UPRISE), il retriever funge da **Task Adapter universale**, in quanto **adatta l'input** (attraverso la concatenazione del prompt recuperato) al compito specifico, lasciando l'LLM congelato.

In pratica, l'adattamento al compito non avviene modificando i pesi del modello, ma ottimizzando l'istruzione (il prompt) data al modello.

¹ Cheng, D., Huang, S., Bi, J., Zhan, Y., Liu, J., Wang, Y., ... & Zhang, Q. (2023, December). Uprise: Universal prompt retrieval for improving zero-shot evaluation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing* (pp. 12318-12337).

Fattori Chiave per l'Efficacia del Retrieval



- Il recupero efficiente di documenti pertinenti è cruciale per il successo dei sistemi RAG. I fattori chiave da considerare includono:
 1. **Retrieval Source**
 2. **Indexing Optimization**
 3. **Query Optimization**
 4. **Embedding**

Fattori Chiave per l'Efficacia del Retrieval



- 1. Retrieval Source:

A. Data Structure

Il tipo e la struttura dei dati esterni influenzano l'arricchimento della LLM:

- **Dati Non Strutturati:** Le fonti più comuni (es. corpus di testo, Wikipedia).
- **Dati Semi-Strutturati:** Dati che combinano testo e tabelle (es. PDF), che presentano difficoltà nello splitting e nella ricerca di similarità semantica.
 - ❖ Approcci: Sfruttare le capacità di codice dell'LLM (Text-2-SQL) o trasformare le tabelle in testo.
- **Dati Strutturati:** Knowledge Graph (KG) verificati, che offrono informazioni più precise ma richiedono sforzi aggiuntivi per costruzione e manutenzione.
- **Contenuto Generato da LLM:** Tendenza a utilizzare il contenuto generato dal modello stesso per l'arricchimento, spesso ottenendo risposte più accurate.

Fattori Chiave per l'Efficacia del Retrieval



- 1. Retrieval Source:

B. Granularità del Retrieval (Retrieval Granularity)

Il livello di scomposizione, la dimensione, delle unità da recuperare:

- **Granularità «Grossolana» (coarse)**: Fornisce contesto più ampio, ma rischia di introdurre ridondanze o contenuto fuorviante (noise).
- **Granularità «Fine»**: Riduce il noise, ma aumenta l'onere del retrieval e non garantisce l'integrità semantica completa.
- **Adattabilità**: La granularità (varia da Token a Documento) può essere adattata in base al compito specifico.

Fattori Chiave per l'Efficacia del Retrieval



- **2. Indexing Optimization:**

1. **Strategie di Segmentazione (*Chunking*):** La suddivisione dei documenti in blocchi (*chunks*) ottimizza il contesto.

- **Segmentazione per Token Fissi:** Metodo più comune (es. 100, 256, 512 token). Blocchi grandi catturano più contesto ma introducono *noise*; blocchi piccoli riducono il *noise* ma possono perdere contesto.
- **Segmentazione Ricorsiva / Finestra Scorrevole:** Metodi per gestire la troncatura delle frasi e collegare informazioni correlate a livello globale.

2. **Metadati (*Metadata Attachments*):** Arricchire i *chunks* con informazioni aggiuntive per migliorare il *retrieval*.

- **Metadati Estratti:** Informazioni come numero di pagina, nome del file, autore, *timestamp*. Permettono il **filtraggio** della ricerca, per esempio per ottenere un RAG consapevole del tempo (*time-aware RAG*).
- **Metadati Costruiti (Reverse HyDE):** Utilizzo della LLM per generare **domande ipotetiche** a cui il documento risponde. Durante il *retrieval*, la similarità viene calcolata tra la *query* originale e le domande ipotetiche, riducendo il *gap* semantico.

3. **Indice Strutturale (*Structural Index*):** Stabilire una struttura gerarchica per accelerare il recupero e ridurre le **allucinazioni**.

- **Struttura Gerarchica:** Organizzazione *parent-child* (genitore-figlio) con riassunti ai nodi. Aiuta il RAG a determinare rapidamente quali *chunks* estrarre.
- **Indice basato su Knowledge Graph (KG):** Utilizza i KG per definire le **connessioni logiche** tra concetti ed entità, mantenendo la coerenza.

Fattori Chiave per l'Efficacia del Retrieval



- **3. Query Optimization:**

La RAG è limitata dalla dipendenza dalla query originale, che è spesso imprecisa, complessa o ambigua.

1. Query Expansion: Espandere la singola query in query multiple arricchisce il contesto per risposte più pertinenti.

- **Multi-Query:** L'LLM espande la query in formulazioni diverse da eseguire in parallelo, ottimizzando la ricerca.
- **Sub-Query:** Scompone domande complesse in sotto-domande semplici (es. least-to-most) la cui combinazione risolve il problema originale.

2. Query Transformation: Il retrieval avviene usando una query modificata anziché quella originale:

- **Query Rewrite:** L'LLM (o un modello più piccolo) riscrive la query per renderla ottimale per il retrieval.
- **HyDE (Hypothetical Document Embeddings):** L'LLM genera documenti ipotetici (risposte). Il retrieval cerca la similarità di embedding tra le risposte, non tra le query.
- **Step-back Prompting:** La query viene astratta in una domanda concettuale di alto livello (step-back question). Entrambe le domande sono usate per il retrieval.

3. Query Routing: Indirizzare la query al pipeline RAG più adatto in base alle sue caratteristiche:

- **Metadata Router/Filter:** Estrae parole chiave per filtrare i chunks in base ai metadati, restringendo l'ambito.
- **Semantic Router:** Utilizza le informazioni semantiche della query per determinare il percorso corretto.
- **Routing Ibrido:** Combina approcci semantici e basati sui metadati.

Fattori Chiave per l'Efficacia della Generation



- È prassi sconsigliata inserire tutto il contesto recuperato direttamente nella LLM per la generazione, a causa della ridondanza e del rischio che il modello possa perdersi qualche dettaglio importante.
- L'ottimizzazione della fase di Generazione nella RAG si basa su due possibili aggiustamenti chiave: l'ottimizzazione del contesto da dare all'LLM oppure effettuare il Fine-Tuning dell'LLM (ovvero, fornire conoscenza aggiuntiva in domini specifici e consentire alla LLM di adattarsi a formati/stili di risposta particolari).
- **Ottimizzazione del contesto:** pulizia e l'ottimizzazione del contesto recuperato.
 - **Reranking:** Riordinare i chunks recuperati per mettere in evidenza i risultati più pertinenti all'inizio, agendo da filtro e potenziatore (attraverso metodi basati su regole (es. rilevanza, diversità) o su modelli dedicati (es. BERT, modelli GPT).
 - **Selezione e Compressione del Contesto:** Un contesto eccessivo riduce la percezione delle informazioni chiave da parte della LLM.
 - ❖ *Compressione:* Utilizzo di SLM (Small Language Models) per rimuovere token non importanti dal prompt o addestramento di estrattori/condensatori.
 - ❖ *Filtraggio:* Riduzione del numero di documenti tramite filtraggio e ordinamento, oppure tramite l'autocritica dell'LLM sul contenuto recuperato per valutarne la pertinenza.

Oltre al retrieval singolo, la RAG può utilizzare processi di augmentation (accrescimento della conoscenza dell'LLM) avanzati:

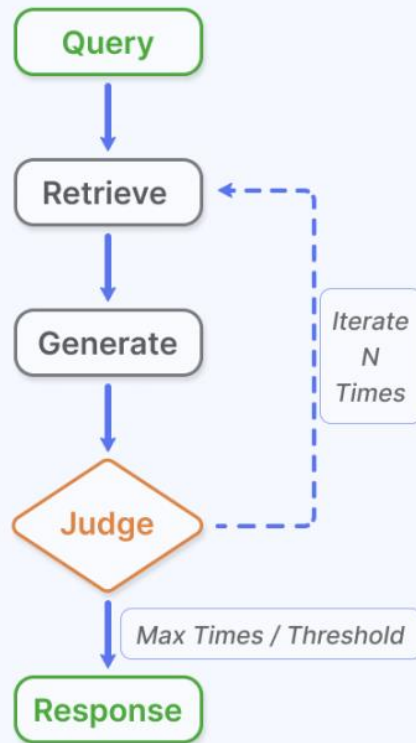
- **Retrieval Iterativo:** Alterna continuamente tra retrieval e generazione. Ad ogni passaggio, il contesto recuperato diventa più ricco e mirato.
- **Retrieval Ricorsivo:** Scomponi la query complessa in sotto-problemi più semplici e continua a risolverli singolarmente tramite retrieval e generazione (funziona come un divide et impera).
- **Retrieval Adattivo:** Determina autonomamente quando è necessario cercare conoscenza esterna e quando fermare il processo di retrieval, utilizzando token speciali generati dalla LLM per il controllo.

Architetture di Retrieval



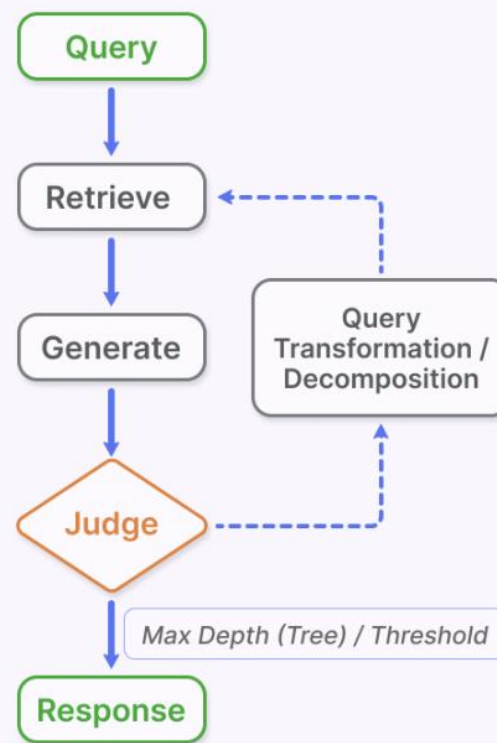
ITERATIVE

Provide more context information



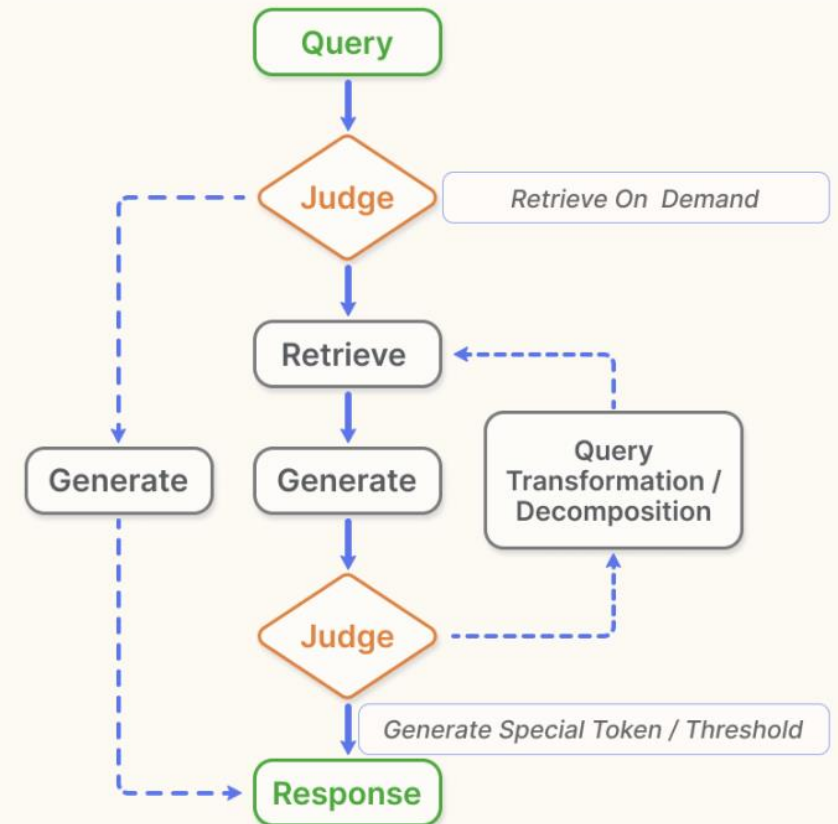
RECURSIVE

Break down complex problems step by step



ADAPTIVE

Flexible and active control of retrieval and generation



RAG in contesti reali



- Perché è utile?
 - Colma il gap tra linguaggio naturale e dati strutturati.
 - Rende gli LLM aggiornati e specifici rispetto a un dominio.
 - Permette di automatizzare query e analisi senza conoscere la sintassi tecnica (SQL, Elasticsearch, ecc.).
- Casi d'uso reali
 - **Customer Support:** LLM che consulta documentazione aziendale per rispondere a clienti.
 - **Ricerca scientifica:** LLM che recupera articoli da database e li sintetizza.
 - **Compliance & Legal:** LLM che trova normative rilevanti e genera riassunti contestualizzati.
 - **Healthcare:** supporto ai medici con retrieval di linee guida e studi aggiornati.
 - **Intelligence:** LLM che genera query su Elasticsearch/SQL per filtrare dashboard e report in base alla domanda dell'analista.

RAG per dati numerici e testuali per Big Data



Un sistema RAG deve recuperare informazioni da diverse fonti e usarle per far rispondere un LLM in modo corretto.

Quando il sistema deve gestire **testo** e **numeri**, la logica rimane la stessa, ma cambia *dove* vengono recuperati i dati.

Possiamo dividere tutto in **3 fasi**.

RAG per dati numerici e testuali per Big Data



1) Indicizzazione – Preparare i dati

- Il sistema raccoglie i dati grezzi e li salva in modo che possano essere recuperati facilmente.
- **Dati testuali** → vengono convertiti in *embedding* e salvati in un **Vector Database** (es. commenti, descrizioni, articoli) come **Chroma, Qdrant**.
- **Dati numerici o metriche** → vengono calcolati o aggregati (medie, somme, conteggi, trend), con ad esempio **Apache Spark**, e salvati in un **database analitico** (es. **Elasticsearch**, time-series DB).
- **Obiettivo:** Organizzare i dati così che possano essere cercati velocemente in seguito.

RAG per dati numerici e testuali per Big Data



2) Recupero – Capire cosa serve e cercarlo nel posto giusto

Quando l'utente fa una domanda, il sistema deve prima **capire il tipo di richiesta**:

Se la domanda riguarda testo

- (es: “Mostrami i commenti negativi degli ultimi clienti”)
→ si usa il **Vector DB** per trovare i passaggi più rilevanti.

Se la domanda riguarda numeri

- (es: “Quante transazioni ci sono state oggi?”)
→ si interroga il **database analitico** per ottenere la metrica esatta. (Ad esempio, qui la query potrebbe essere trasformata da un LLM da linguaggio naturale ad una query Elasticsearch)
- A volte la domanda richiede **entrambi**:
(es: “Perché oggi ci sono state così tante cancellazioni?”)

In questo caso il sistema recupera:

- una **metrica** (es: “Cancellazioni oggi: 482”)
- alcuni **contesti testuali** (es: “Molti utenti segnalano problemi di pagamento”)
- **Obiettivo**: Preparare un pacchetto di informazioni— da dare all'LLM.

3) Generazione – L'LLM risponde usando i dati recuperati

Il sistema costruisce un prompt che contiene:

- la domanda dell'utente
- i testi trovati
- i dati numerici recuperati

E lo passa all'LLM.

L'LLM combina tutto e genera una risposta chiara, per esempio:

“Oggi ci sono state 482 cancellazioni. Molti utenti hanno lamentato problemi con il sistema di pagamento, il che spiega l'aumento rispetto alla media settimanale.”

Se richiesto, il sistema può anche produrre un link o una visualizzazione (**Grafana/Kibana**) per mostrare grafici o trend.

Obiettivo: Trasformare dati grezzi (numeri + testo) in una risposta comprensibile.

RAG per dati numerici e testuali per Big Data



Inoltre, un LLM può controllare una dashboard.

L'LLM può:

- cambiare filtri
- cambiare l'intervallo temporale
- selezionare metriche
- cambiare la visualizzazione
- aprire specifici pannelli
- generare link dinamici
- aggiornare query SQL o Elasticsearch
- creare nuove visualizzazioni

Non agisce da solo sulla dashboard, ma produce **comandi** o **query** che la dashboard usa per aggiornare la vista. L'LLM diventa una interfaccia naturale che traduce la richiesta dell'utente in azioni sulla dashboard.

RAG per dati numerici e testuali per Big Data



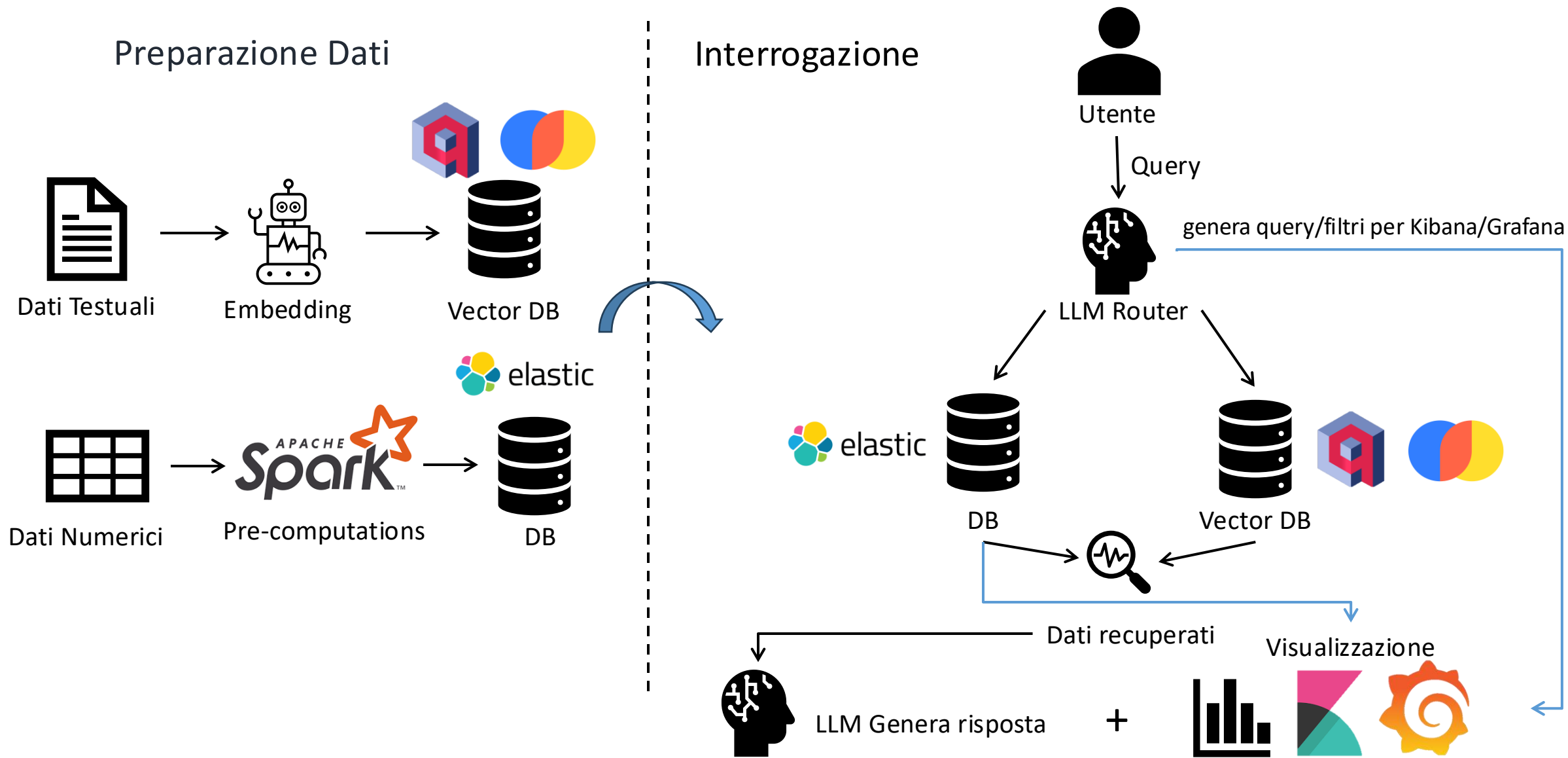
Inoltre, un LLM può controllare una dashboard.

Processo riepilogativo:

1. L'utente fa una richiesta in linguaggio naturale.
2. L'LLM la interpreta e capisce cosa deve essere fatto.
3. L'LLM, come router, sceglie la funzione giusta tra quelle disponibili.
4. Genera un comando strutturato (function call / JSON).
5. Il backend esegue questo comando sulla dashboard tramite API.
6. La dashboard si aggiorna in tempo reale.

[1] Utente → [2] LLM Router → [3] Backend/Tools → [4] Dashboard

RAG per dati numerici e testuali per Big Data





LLM + MongoDB + Grafana: un approccio RAG per l'analisi dei dati

*Un sistema ibrido per generare, eseguire e visualizzare query
in linguaggio naturale*

1. Gli utenti non tecnici faticano a scrivere query MongoDB.
 2. Visualizzare i dati già filtrati è d'impatto e aiuta gli analisti nello studio e nell'analisi.
-  Obiettivo: permettere **interrogazioni in linguaggio naturale** e visualizzare i dati in base alla richiesta dell'analista su una **dashboard strutturata**.

Componenti principali



- **Gemini 2.5 Pro (LLM):** genera pipeline di aggregazione MongoDB a partire da una richiesta in linguaggio naturale e una risposta descrittiva dei documenti recuperati.
- **MongoDB Atlas Vector Search:** recupera documenti rilevanti.
- **RAG (Retrieval-Augmented Generation):** i documenti recuperati vengono passati all'LLM per ricevere una prima risposta descrittiva.
- **Flask API:** interfaccia REST tra LLM, MongoDB e Grafana.
- **Grafana + Infinity Plugin:** visualizza i risultati in tempo reale in base alla richiesta dell'utente.

Workflow



1. L'utente invia un prompt naturale.
2. L'LLM genera:
 1. una risposta iniziale basata sui documenti recuperati (RAG),
 2. una **query MongoDB valida** (pipeline JSON).
3. Flask esegue la query.
4. I risultati vengono salvati e resi disponibili a Grafana via endpoint JSON.

Retrieval Augmented Generation (RAG)



- Si calcola l'embedding del prompt con il modello di embedding *all-MiniLM-L6-v2* (stesso modello usato per l'indicizzazione dei documenti).
- Si effettua \$vectorSearch su MongoDB per recuperare i documenti più simili al prompt.
- L'LLM usa questi documenti come **contesto informativo**.
- Restituisce:
 - una **risposta naturale** che riassume i risultati
 - la **pipeline MongoDB** da eseguire.



Cluster



Overview

Data Explorer

Real Time



Cluster Metrics



Query Insights



Performance Advisor

Online Archive

Command Line Tools

Infrastructure as Code

SHORTCUTS

Search & Vector Search

PROJECT
ConversationalAnaly...CLUSTER
crimes[← Back to Clusters](#)

Overview

crimes

VERSION

8.0.16

REGION

AWS Frankfurt (eu-central-1)

CLUSTER TIER

Free

Connect

Configuration



TAGS

Use tags to efficiently label and categorize your clusters. Any tags you apply will display here.

[Learn more about tagging.](#)

ADD TAG

REGION Frankfurt (eu-central-1)

ac-ox4r... shard-00-00.v68lb...

SECONDARY

ac-ox4rh... shard-00-01.v68lb0...

PRIMARY

ac-ox4r... shard-00-02.v68lb...

SECONDARY

This is a Free Tier Cluster

If you need a database that's better for high-performance production applications, upgrade to a dedicated cluster.

Upgrade

Operations R: 0.03 W: 0.07/s



Last 6 Hours

Logical Size 166.3 MB



Last 30 Days

Connections 13



Last 6 Hours



Documenti in MongoDB



crime_demo.crime_records

STORAGE SIZE: 72.4MB LOGICAL DATA SIZE: 49.65MB TOTAL DOCUMENTS: 10000 INDEXES TOTAL SIZE: 588KB

Find

Indexes

Schema Anti-Patterns 0

Aggregation

Search Indexes

[Generate queries from natural language in Compass](#)

INSERT DOCUMENT

Filter

Type a query: { field: 'value' }

Reset

Apply

Options

QUERY RESULTS: 1-20 OF MANY

```
_id: ObjectId('6900c4b9d3d0ac7e459e8a70')
Report_ID : 200513040
Date : "8/16/2020 0:00"
Location : "GAFFEY"
Area : "Harbor"
Crime_Type : "ATTEMPTED ROBBERY"
Victim_Age : 42
Victim_Sex : "F"
Premise : "BUS STOP"
Weapon : "STRONG-ARM (HANDS, FIST, FEET OR BODILY FORCE)"
Status : "Adult Other"
▶ embedding : Array (384)
DateObj : 2020-08-16T00:00:00.000+00:00
```

```
1 _id: ObjectId('6900c4b9d3d0ac7e459e8a71')
2 Report_ID : 201915946
3 Date : "10/20/2020 0:00/"
4 Location : "9800 N SEPULVEDA"
5 Area : "Mission/"
```

ObjectId
Int32
String
String
String



PREVIOUS

1-20 of many results

NEXT



Crimini di tipo « Tentata Rapina »



LLM-MongoDB Query Interface

show me all crime type ATTEMPTED ROBBERY

Run Query

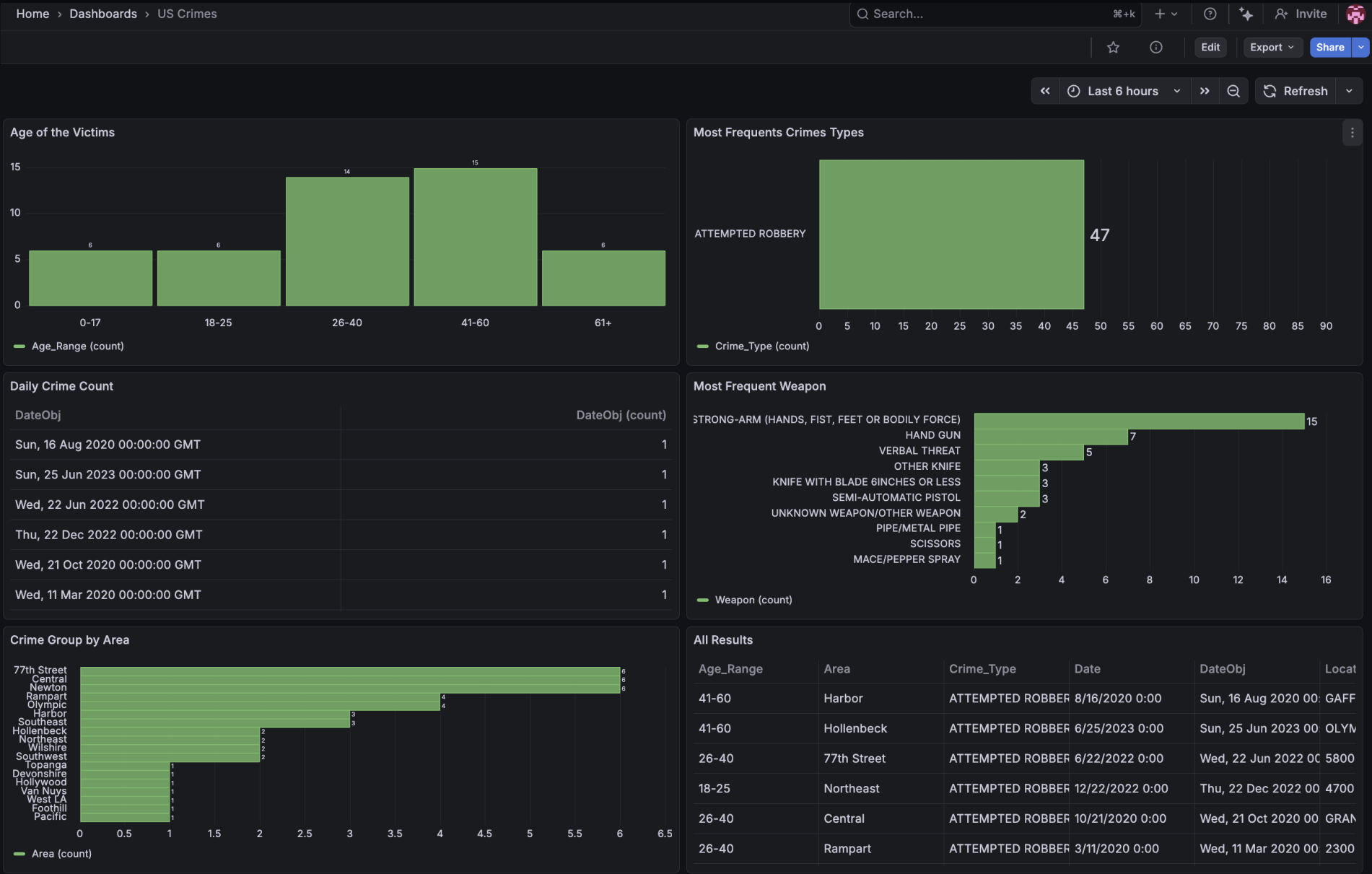
Results:

The dataset includes 47 reported cases of 'ATTEMPTED ROBBERY'. Now we can see all details in our dashboard :)

► Show full results (JSON)

```
[{'$match': {'Crime_Type': 'ATTEMPTED ROBBERY'}}]
```

Crimini di tipo « Tentata Rapina »



Crimini nell'area di Harbor



LLM-MongoDB Query Interface

show me all crime in Harbor Area

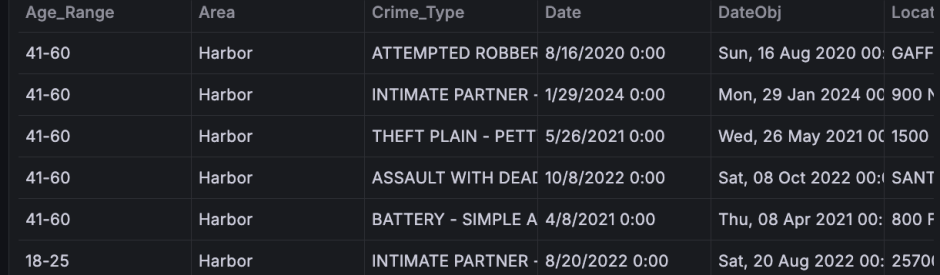
Run Query

Results:

The query found 427 crime incidents in the Harbor area. Now we can see all details in our dashboard :)

► Show full results (JSON)

```
[{'$match': {'Area': 'Harbor'}}]
```



Crimini con pistola come arma



LLM-MongoDB Query Interface

show me all crime with HAND GUN as weapon

Run Query

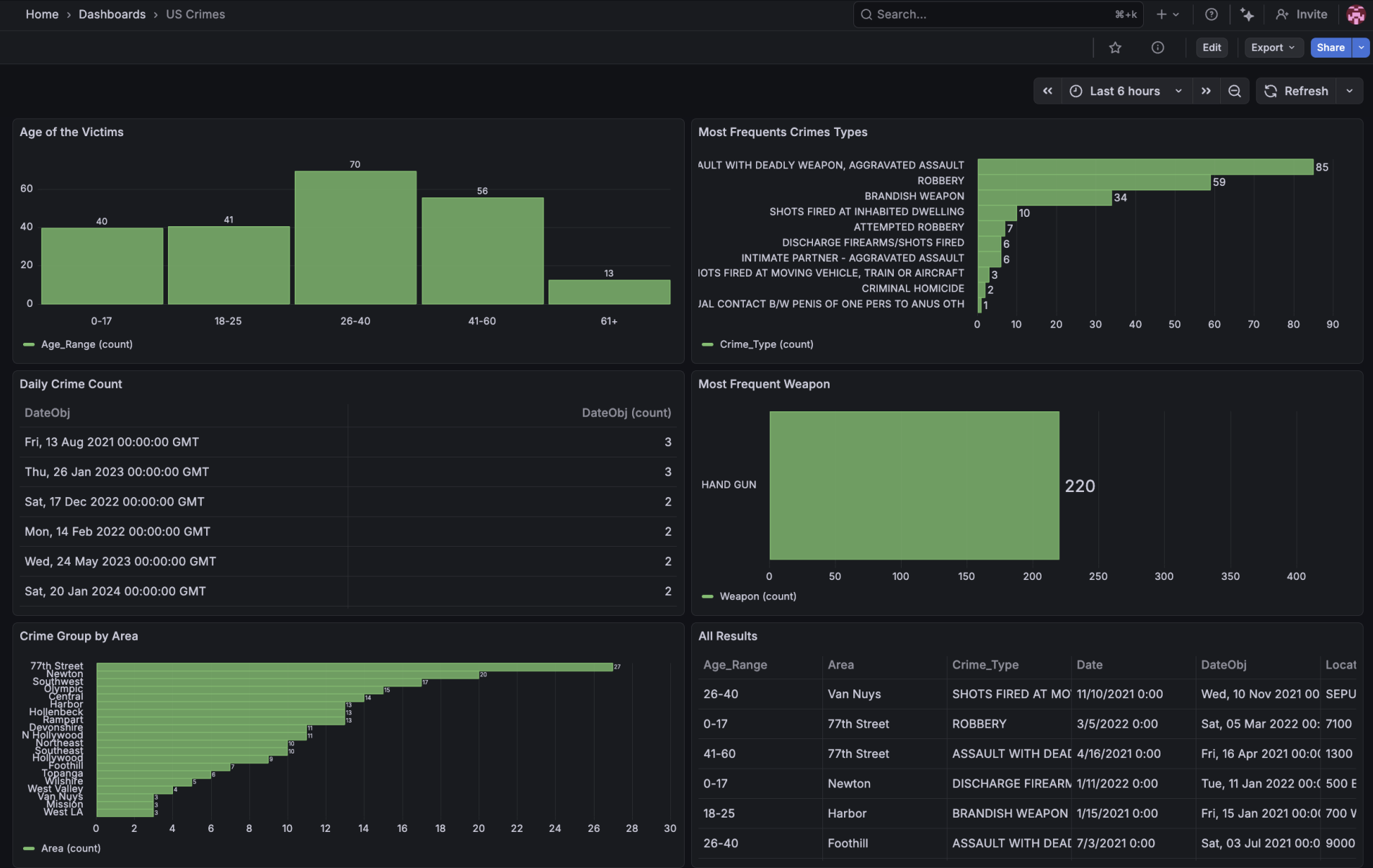
Results:

The query returned 220 crime incidents where a hand gun was used as the weapon.
Now we can see all details in our dashboard :)

► Show full results (JSON)

```
[{'$match': {'Weapon': 'HAND GUN'}}]
```

Crimini con pistola come arma



Crimini con arresto



LLM-MongoDB Query Interface

show me all crime with status Adult Arrest

Run Query

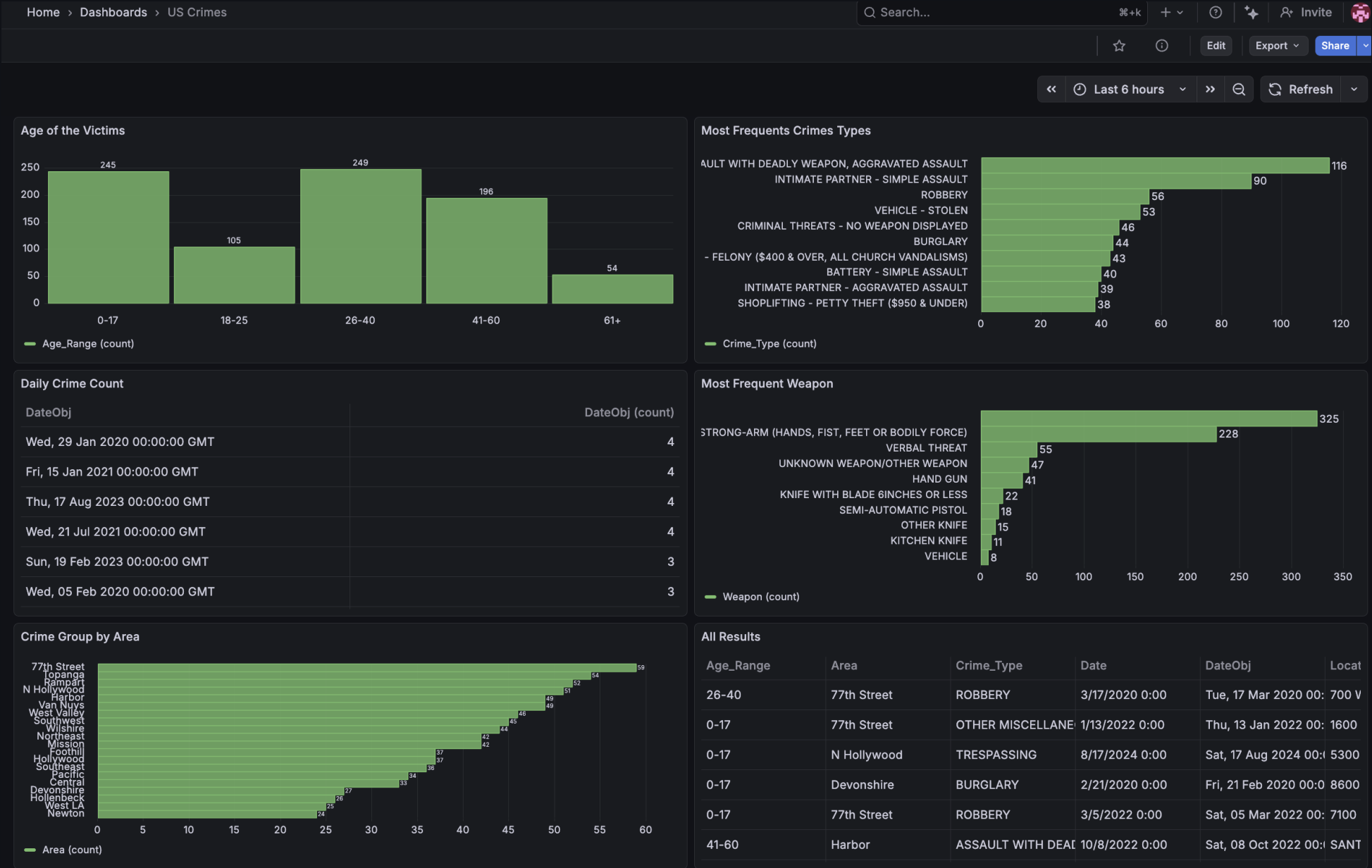
Results:

The query found 849 crime incidents that resulted in an adult arrest. Now we can see all details in our dashboard :)

► Show full results (JSON)

```
[{'$match': {'Status': 'Adult Arrest'}}]
```

Crimini con arresto



Conclusioni



- L'LLM diventa un “traduttore semantico” tra linguaggio naturale e query MongoDB.
- L'analisi dei dati diventa accessibile anche a utenti non tecnici.
- Grafana fornisce una dashboard interattiva e in tempo reale in base alle esigenze dell'analista.
- *“L'intelligenza artificiale non sostituisce l'analista, ma gli offre un nuovo linguaggio per dialogare con i dati.”*